



Get started Prepare ▾

Get full access



Blog Menu

Series Contents

30 Basic to Advanced React Interview Questions with Solutions

30 React interview questions and solutions, covering basic to advanced topics. Ideal for developers preparing for their next job interview in 2025



Starter

[#react](#)[#interviews](#)

GreatFrontEnd Team

11 min read Jul 1, 2025

Copy link

Share



30 Basic to Advanced React Interview Questions with Solutions

Preparing for React developer interviews can be daunting, especially with so many questions to choose from. With a wealth of resources available, it's crucial to identify the most relevant topics that will give you the best return on your time investment.

React interviews often emphasize a mix of core concepts and practical skills, such as state management, hooks, performance optimization, etc. To enhance your preparation, focus on the most commonly asked questions and those that integrate multiple skills for a comprehensive understanding.

In this list, we've compiled 30 essential React interview questions that are essential for your success.

If you're looking for more in-depth React interview preparation materials, also check out these resources:

- [React Interview Playbook](#)

- [Practice React coding interview questions](#)
- [Practice React quiz interview questions](#)

Basic Level Questions

1. What is React?

React is an open-source JavaScript library developed by Facebook for building user interfaces, particularly single-page applications (SPAs). It allows developers to create reusable UI components that manage their own state. Key benefits include a component-based structure, efficient updates with the virtual DOM, declarative UI for readability, and strong community support.



[Read more about it](#)

2. What is JSX?

JSX stands for JavaScript XML and is a syntax extension for JavaScript that lets you write HTML-like code within JavaScript. It simplifies creating React components. JSX is transformed into JavaScript function calls, usually by Babel. For example, `<div>Hello, world!</div>` becomes `React.createElement('div', null, 'Hello, world!')`.



[Read more about it](#)

3. Difference Between React Node, Element, and Component

A React Node is any renderable unit in React, like an element, string, number, or `null`. A React Element is an immutable object describing what to render, created with `JSX` or

`React.createElement` . A React Component is a function or class that returns React Elements, allowing for reusable UI pieces.

[Read more about it](#)

4. What are fragments in React?

Fragments allow grouping multiple elements without adding extra nodes to the DOM, helping keep the markup clean and efficient.

```
1  return (  
2    <>  
3      <h1>Title</h1>  
4      <p>Description</p>  
5    </>  
6  );
```

[Read more about it](#)

5. What is the virtual DOM?

The virtual DOM is a lightweight copy of the real DOM. React uses it to optimize rendering by updating only the parts of the DOM that have changed, rather than re-rendering the entire tree.

[Read more about it](#)

6. What is the purpose of the key Prop in React?

The key prop in React uniquely identifies elements in a list, helping React optimize rendering by efficiently updating and reordering items. Without unique keys, React may unnecessarily re-render elements, leading to performance issues and bugs.

```
{
  items.map((item) => <ListItem key={item.id} value={item.value} />);
}
```

[Read more about it](#)

7. What are the implications of using array indices as keys in React?

Using array indices as keys in React can lead to performance problems and unexpected behavior. When the order of items in an array changes, React might struggle to accurately determine which items have been modified. This can result in unnecessary re-renders or incorrect updates to the UI. To ensure efficient DOM management, it is advisable to use unique identifiers for keys instead of relying on array indices.

[Read more about it](#)

Intermediate Level Questions

8. What are the rules of React hooks?

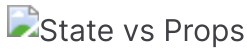
React hooks must be called at the top level of a function, never inside loops, conditions, or nested functions. They should only be called from React function components or custom hooks. These rules help maintain correct state and lifecycle behavior.

[Read more about it](#)

9. How do state and props differ in React?

- **State:** State is an internal data structure managed within a component. It is mutable, meaning that it can be updated and changed over time, allowing the component to respond to user interactions or other events dynamically.

- **Props:** Props, short for properties, are external data passed from parent components to child components. They are immutable within the child component, meaning that once set, they cannot be altered by the child. This ensures a clear flow of data and helps maintain the integrity of the component hierarchy.



[Read more about it](#)

10. What are controlled components?

Controlled components are form elements whose values are controlled by React state, allowing for more predictable behavior and easier validation.

```
1  function MyForm() {  
2    const [inputValue, setInputValue] = useState('');  
3  
4    const handleChange = (e) => {  
5      setInputValue(e.target.value);  
6    };  
7  
8    return <input type="text" value={inputValue} onChange={handleChange} />;  
9  }
```

[Read more about it](#)

11. Explain hooks in React.

Hooks are functions that allow developers to use state and other React features without writing a class. Common hooks include `useState` for managing state and `useEffect` for side effects.

12. What distinguishes `useEffect` from `useLayoutEffect` in React?

`useEffect` and `useLayoutEffect` are both hooks utilized for managing side effects in React functional components, but they differ in terms of execution timing:

- `useEffect` : This hook runs asynchronously after the DOM has been updated and painted. It is well-suited for operations like data fetching or setting up subscriptions, as it does not block the rendering process.
- `useLayoutEffect` : In contrast, `useLayoutEffect` executes synchronously immediately after DOM mutations but before the browser has a chance to paint. This makes it ideal for tasks that require immediate access to the DOM, such as measuring element sizes or synchronizing the UI with the current DOM state.

Code Example:

```
1  import React, { useEffect, useLayoutEffect, useRef } from 'react';
2
3  function Example() {
4    const ref = useRef();
5
6    useEffect(() => {
7      console.log('useEffect: Runs after DOM paint');
8    });
9
10   useLayoutEffect(() => {
11     console.log('useLayoutEffect: Runs before DOM paint');
12     console.log('Element width:', ref.current.offsetWidth);
13   });
14
15   return <div ref={ref}>Hello</div>;
16 }
```

[Read more about it](#)

13. What does the dependency array of `useEffect` affect?

The dependency array of `useEffect` controls when the effect re-runs:

- If it's empty, the effect runs only once after the initial render.

- If it contains variables, the effect re-runs whenever any of those variables change.
- If omitted, the effect runs after every render.

[Read more about it](#)

Advanced Level Questions

14. What is Redux?

Redux is a predictable state management library often used with React to manage application state through actions and reducers, promoting a unidirectional data flow.

15. Explain prop drilling.

Prop drilling occurs when you pass data through multiple layers of components, even if some intermediate components don't need that data. This can make your code cumbersome and harder to maintain.

```
1  // ParentComponent.js
2  import React from 'react';
3  import ChildComponentA from './ChildComponentA';
4
5  function ParentComponent() {
6    const data = 'Hello from Parent';
7    return <ChildComponentA data={data} />;
8  }
9
10 // ChildComponentA.js
11 import React from 'react';
12 import ChildComponentB from './ChildComponentB';
13
14 function ChildComponentA({ data }) {
15   return <ChildComponentB data={data} />;
16 }
```

```
17
18 // ChildComponentB.js
19 import React from 'react';
20 import ChildComponentC from './ChildComponentC';
21
22 function ChildComponentB({ data }) {
23   return <ChildComponentC data={data} />;
24 }
25
26 // ChildComponentC.js
27 import React from 'react';
28
29 function ChildComponentC({ data }) {
30   return <h1>{data}</h1>;
31 }
32
33 export default ChildComponentC;
```

In the example above, `data` is passed from `ParentComponent` to `ChildComponentC` through `ChildComponentA` and `ChildComponentB`, which don't use it. This unnecessary passing exemplifies prop drilling. To avoid this, consider using the Context API for more efficient data sharing.

16. What are higher-order components (HOCs)?

HOCs are functions that take a component and return a new component, allowing for code reuse and abstraction of common functionality.

```
1 function withLogging(WrappedComponent) {
2   return function EnhancedComponent(props) {
3     console.log('Rendering:', WrappedComponent.name);
4     return <WrappedComponent {...props} />;
5   };
6 }
```

[Read more about it](#)

17. Describe lazy loading in React.

Lazy loading is an optimization technique where components or modules are loaded only when they are needed, improving application performance.

```
1  const LazyComponent = React.lazy(() => import('./LazyComponent'));
2
3  function App() {
4    return (
5      <React.Suspense fallback={<div>Loading...</div>}>
6        <LazyComponent />
7      </React.Suspense>
8    );
9  }
```

[Read more about it](#)

18. What is Context API?

The Context API provides a way to share values (like global state) between components without having to explicitly pass props through every level of the tree.

[Read more about it](#)

19. How do you optimize performance in React applications?

Techniques include:

- Using `React.memo` for functional components.
- Implementing `shouldComponentUpdate` for class components.
- Utilizing lazy loading for code splitting.

[Read more about it](#)

20. Explain error boundaries in React.

Error boundaries are special components that catch JavaScript errors anywhere in their child component tree, log those errors, and display a fallback UI instead of crashing the whole application.

```
1  class ErrorBoundary extends React.Component {
2    constructor(props) {
3      super(props);
4      this.state = { hasError: false };
5    }
6
7    static getDerivedStateFromError(error) {
8      return { hasError: true };
9    }
10
11   componentDidCatch(error, info) {
12     console.error('Error caught:', error);
13   }
14
15   render() {
16     if (this.state.hasError) {
17       return <h1>Something went wrong.</h1>;
18     }
19     return this.props.children;
20   }
21 }
```

[Read more about it](#)

21. What are custom hooks?

Custom hooks allow developers to extract component logic into reusable functions, promoting cleaner code and better organization.

```
1  function useFetch(url) {
2    const [data, setData] = useState(null);
```

```
3
4   useEffect(() => {
5     fetch(url)
6       .then((response) => response.json())
7       .then((data) => setData(data));
8   }, [url]);
9
10  return data;
11 }
```

22. How does React handle forms?

Forms can be controlled or uncontrolled; controlled forms use state to manage input values while uncontrolled forms rely on DOM elements directly.

[Read more about it](#)

23. What is server-side rendering (SSR)?

Server-side rendering (SSR) involves rendering components on the server before sending fully rendered HTML to clients, improving initial load times and SEO through efficient hydration processes.



[Read more about it](#)

24. How do you test React applications?

Testing React applications can be done using Jest and React Testing Library. Jest serves as the testing framework while React Testing Library provides utilities for testing components similarly to user interactions.

[Read more about it](#)

25. Discuss synthetic events in React.

Synthetic events are cross-browser wrappers around native events that provide consistent behavior across different browsers while maintaining performance optimizations.

26. Describe `useReducer` hook in React.

The `useReducer` hook is an alternative to `useState` for managing complex state logic by using reducers similar to Redux patterns.

```
1  const initialState = { count: 0 };
2
3  function reducer(state, action) {
4    switch (action.type) {
5      case 'increment':
6        return { count: state.count + 1 };
7      case 'decrement':
8        return { count: state.count - 1 };
9      default:
10       throw new Error();
11    }
12  }
13
14  function Counter() {
15    const [state, dispatch] = useReducer(reducer, initialState);
16
17    return (
18      <>
19        Count: {state.count}
20        <button onClick={() => dispatch({ type: 'increment' })}></button>
21        <button onClick={() => dispatch({ type: 'decrement' })}></button>
22      </>
23    );
24  }
```

[Read more about it](#)

27. Explain what React hydration is.

Hydration involves attaching event listeners and making server-rendered HTML interactive on the client side. After server-side rendering, React initializes dynamic behavior by attaching event handlers.



[Read more about it](#)

28. What are some React anti-patterns?

React anti-patterns are practices that can lead to inefficient or hard-to-maintain code. Common examples include:

- Directly mutating state instead of using `setState`
- Using `componentWillMount` for data fetching
- Overusing `componentWillReceiveProps`
- Not using keys in lists
- Excessive inline functions in render
- Deeply nested state

[Read more about it](#)

29. Explain how you would implement routing in a React application using React Router?

You can implement routing using `react-router-dom`. Here's an example:

```
1  import { BrowserRouter as Router, Route, Switch } from 'react-router-dom';
2  function App() {
3    return (
4      <Router>
5        <Switch>
```

```
6      <Route path="/" exact component={Home} />
7      <Route path="/about" component={About} />
8      <Route path="/contact" component={Contact} />
9    </Switch>
10  </Router>
11  );
12 }
```

30. How do you localize React applications?

Localization typically involves libraries like `react-i18next` or `react-intl`. Set up translation files for different languages and configure the library within your app using provided hooks or components.

```
1  // Example using react-i18next
2  import { useTranslation } from 'react-i18next';
3
4  const MyComponent = () => {
5    const { t } = useTranslation();
6    return <p>{t('welcome_message')}</p>;
7  };
```

[Read more about it](#)

Conclusion

If you're looking for more in-depth React interview preparation materials, also check out these resources:

- [React Interview Playbook](#)
- [Practice React coding interview questions](#)
- [Practice React quiz interview questions](#)

[← Previous](#)[Next →](#)[50 Must-know HTML, CSS and JavaScript Interview Questions by Ex-...](#)[50+ Must-know JavaScript Interview Questions by Ex-interviewers \(2026\)](#)

Related articles

 [50 React.js Interview Questions for Experienced Developers](#)

50 React.js Interview Questions for Experienced Developers

Jun 22, 2025 [#react](#)

50 React JS interview questions for experienced developers. Explore detailed solutions to enhance your preparation for 2025 job interviews



Subscribe to our newsletter

[Notify me](#)[Practice](#)[Study Plans](#)[Get started](#)[1 Week Plan](#)

JavaScript functions

User interface coding

System design

Quiz

Guides

Front End Interview Playbook

Front End System Design Playbook

React Interview Playbook

Behavioral Interview Playbook

1 Month Plan

3 Months Plan

Company

Pricing

Promotions

Roadmap

About

Team

Contact us

Advertise with us New

Become an affiliate

Careers

Blog

Medium

DEV Community



English (US) ▾

© 2026 Codeney Pte Ltd. All rights reserved.

Privacy Policy

Terms of Service